3장
변수로 데이터 저장하기

지금까지 프로그래밍의 개념과 기본 명령어인 입출력 명령어를 배웠습니다. 데이터를 받고 출력하기는 했지만 입력받는 수준의 프로그램을 작성하기에는 한계가 있었습니다. 이 장에서는 데이터를 저장하는 공간인 변수를 배우고 이를 활용해 좀 더 복잡한 프로그램을 작성해 봅니다. 실습 코드를 작성하고 프로그램을 실행해 보면서 여러 입출력 함수의 의미를 생각해본다면 쉽게 학습할 수 있을 것입니다.

3.1

데이터를 저장하는 공간: 변수

3.1.1 변수란

'변수'라는 단어를 많이 들어 봤을 겁니다. 수학에 자주 나오죠. 수학에서 변수는 '여러 값으로 변하는 수'를 의미합니다. b = a + 1처럼 문자와 숫자를 기호로 연결해 계산하는 수식에서 a는 10일 때 b = 10 + 1 = 11이 됩니다. 또는 a가 20이라면 b = 20 + 1 = 21이 되죠. a가 변하면서 b도 변합니다. 이렇게 수식 안에서 여러 값으로 변하는 a와 b를 변수라고 합니다.

프로그래밍에서도 비슷합니다. 코드로 `a = 10`이라고 작성하면 `a`에 `10`을 넣는다는 의미입니다. 바로 실행해 봅시다.

```
a = 10
```

실행하면 어떤 일이 벌어지나요? 아무 일도 일어나지 않습니다. 아랫줄에 `print(a)`를 추가해 다시 실행해 보세요.

```
a = 10
print(a)
```

```
실행결과
10
```

앞에서 왜 결과가 나오지 않았는지 알겠죠? `a`에 값을 넣으라고만 했지 이를 출력하라고 명령하지 않아서 그렇습니다. 이번 실행결과를 보니 어떤가요? `a`에 `10`을 넣었으니 `print(a)`에서 `a`에 들어 있는 `10`이 출력되는 것이 당연하겠죠.

그러면 프로그래밍에서 변수가 어떤 건지 정리해 볼까요? 컴퓨터는 `a = 10`이라는 코드를 읽으면 컴퓨터에서 사용할 수 있는 빈 공간을 찾습니다. 그리고 찾은 공간(그림에서 박스로 표현한 부분)에 `a`라는 이름을 붙이고 값인 10을 넣습니다. 이렇게 하면 컴퓨터에 변수가 만들어집니다.



▲ 그림 3-1 프로그래밍에서 변수의 의미

3.1.2 변수에 값을 저장하고 사용하기

사람이 뇌에서 정보를 기억하듯이 컴퓨터는 '메모리(memory)'라는 장치에서 정보를 기억합니다. 앞에서 변수를 만들 때 찾은 공간이 메모리입니다. 컴퓨터는 메모리에서 빈 공간을 찾아 변수명을 붙이고 값을 넣어 기억하는 것이죠. 이처럼 변수에 값을 넣어 기억하는 행위를 저장이라고 합니다. 예를 들어, 컴퓨터가 `a = 10`이라는 코드를 읽으면 메모리 공간을 찾아 `a`라는 이름으로 변수를 만들고 이 변수에 10을 저장합니다. 그리고 변수를 만들 때 처음 저장하는 값을 초깃값이라고 합니다. 따라서 `a` 변수의 초깃값은 10이 됩니다. 파이썬에서 변수를 만들려면 반드시 초깃값을 저장해야 합니다. 이렇게 변수를 만들어 초깃값을 저장하는 작업을 변수를 정의한다고도 합니다.

변수에 값을 저장하고 나서 이를 다시 사용하려면 값 자체인 10이 아니라 변수명 `a`로 불러내야 합니다. 10이라는 값에 `a`라는 이름이 붙었다고 생각하면 됩니다. 그래서 10을 출력할 때 `print(10)`이 아니라 `print(a)`로 코드를 작성합니다. 즉, 여기서 `print(a)`는 문자 `a`를 출력하라는 명령이 아니라 변수 `a`에 저장된 값을 출력하라는 명령입니다.

NOTE print(a)와 print('a')의 차이

2장에서 배운 `print('a')`와 3장에서 배우는 `print(a)`는 매우 큰 차이가 있습니다. 바로 작은따옴표가 있고 없고의 차이죠. 2장에서 작은따옴표가 양옆에 붙으면 문자로 인식한다고 했으므로 `print('a')`는 문자 a를 그대로 출력합니다. 작은따옴표가 붙지 않은 `print(a)`는 어떻게 될까요? 파이썬에서 작은따옴표나 큰따옴표가 붙지 않은 문자는 자동으로 변수로 인식합니다. 따라서 다음과 같이 변수 `a`에 저장된 10이 출력됩니다.

```
a = 10
print('a')
print(a)
```

```
실행결과
a
10
```

이 부분이 헷갈리면 오류가 발생합니다. 2장에 나왔던 코드를 다시 봅시다.

```
print(가나다)
```

```
실행결과
NameError: name '가나다' is not defined
```

이 부분에서 발생한 오류를 이제 이해할 수 있을 거예요. 오류 내용을 해석하면 “이름 ‘가나다’는 정의되지 않았다”입니다. 보다시피 문자인 가나다 양옆에 작은따옴표가 없어서 컴퓨터가 이를 문자가 아닌 변수로 인식했기 때문입니다. 즉, 이름이 `'가나다'`인 변수가 만들어지지 않았다는 것이죠. 컴퓨터는 `'가나다'`를 변수라고 인식했는데 해당 이름의 변수가 존재하지 않으니 컴퓨터가 명령을 이해하지 못해 오류가 발생한 겁니다.

앞으로 이런 `NameError`를 만나면 당황하지 말고 변수명과 변수에 들어 있는 값, 문자 양옆에 작은따옴표를 빠뜨리지 않았는지 확인하세요.

변수에 문자 저장하기
프로그래밍에서는 수학에서와 달리 변수에 숫자뿐만 아니라 문자도 넣을 수 있습니다.

```
b = '학생'
print(b)
```

```
실행결과
학생
```

`b`라는 변수에 문자 '학생'을 저장했습니다. 앞에서 설명했듯이 문자는 반드시 양옆에 작은따옴표나 큰따옴표가 있어야 문자로 인식합니다. 코드를 실행해 보니 예상한 대로 '학생'이 출력됩니다. 이 코드를 다음과 같이 바꿔 봅시다.

```
b = '학생'
c = '고등'
print(b + c)
```

```
실행결과
학생고등
```

변수 `c`에 문자 '고등'을 저장하고 `b`와 `c`를 합쳐 출력하니 '학생고등'이 출력됩니다. 2장에서 '문자 + 문자'는 공백 없이 문자가 합쳐져서 출력된다고 배웠습니다. `b`와 `c`에 들어 있는 값이 모두 문자이므로 문제없이 출력됩니다. 그런데 단어가 이상합니다. 고등학생이 맞는 단어죠?

이는 `print()`에서 먼저 등장한 변수를 출력하고, 그 다음 변수를 출력하기 때문입니다. `print(b + c)`는 명령어 순서대로 `b`에 들어 있는 '학생'을 먼저 출력하고, 그다음 `c`에 들어 있는 '고등'을 출력합니다. 제대로 고등학생을 출력하려면 어떻게 할까요? 명령어 순서를 바꾸면 됩니다.

```
b = '학생'
c = '고등'
print(c + b)
c = '대'
print(c + b)
```

```
실행결과
고등학생
대학생
```

첫 번째는 고등학생이 출력되지만, 두 번째는 대학생이 출력됩니다. 처음에 변수 `c`에 문자 '고등'을 저장합니다. 그리고 네 번째 줄에서 `c`에 새로운 문자 '대'를 저장합니다. 이렇게 하면 처음 `c`에 넣은 값은 사라지고 새롭게 넣은 값이 `c`에 저장됩니다. 즉, 변수에는 마지막으로 저장한 값만 남고 이전 값은 없어집니다. 이처럼 변수는 하나의 값만 기억(저장)합니다.

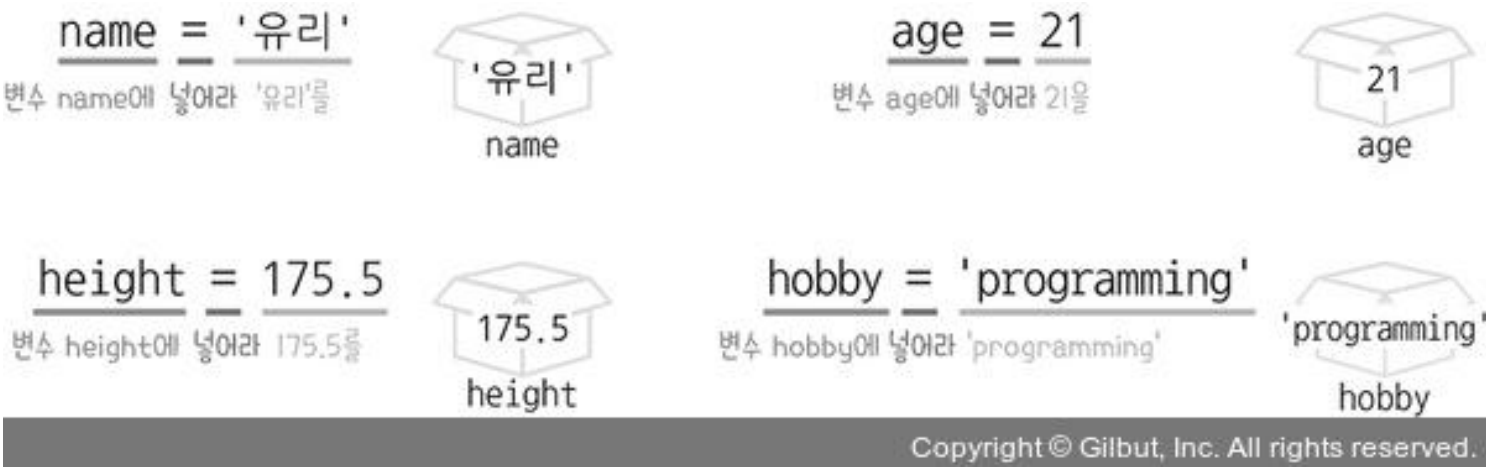
조금 더 일반적인 예를 살펴봅시다.

```
name = '유리'
height = 175.5
age = 21
hobby = 'programming'
print(name, height, age, hobby)
```

```
실행결과
유리 175.5 21 programming
```

이 코드는 각 변수에 이름, 키, 나이, 취미를 저장합니다. 그리고 마지막 출력 명령어에서 `name`, `height`, `age`, `hobby` 변수에 저장된 값을 출력합니다. 이때 각 변수를 콤마(`,`)로 연결했습니다. 콤마(`,`)는 2장에서 배웠듯이 여러 요소를 연결하는 역할을 하며, 문자나 숫자 외에 변수를 연결할 때도 사용합니다.

4개의 변수를 만들 때 컴퓨터 내부에서 일어나는 일은 다음과 같습니다.



▲ 그림 3-2 변수 만들기

여기서 핵심은 변수에 들어 있는 값을 사용하고 싶을 때 값인 '유리'를 직접 사용하는 것이 아니라 값이 저장된 `name` 변수를 사용한다는 것입니다.

변수를 어떻게 활용하는지 자세히 알아봅시다. 출력 결과가 `'유리 키는 175.5, 나이는 21, 취미는 programming입니다!'`가 되게 하려면 앞의 코드를 어떻게 수정해야 할까요? 변수에 저장된 부분과 그렇지 않은 부분을 구분해 봅시다.



변수 부분과 문자 그대로 출력되는 부분을 색깔을 다르게 하니 구분되죠? 변수와 문자는 숫자와 문자처럼 서로 다른 요소입니다. 서로 다른 요소는 콤마(,)로 연결해 출력합니다. 따라서 노란색 부분의 변수와 파란색 부분의 문자 사이에 콤마를 넣습니다. 그리고 문자는 양옆에 작은따옴표를 붙여 표현합니다. 요소를 연결하는 콤마 외에 문자(파란색) 안에 포함된 콤마(,)도 있으니 잘 구분해야 합니다.

정리한 내용을 코드로 작성하고 실행해 봅시다.

```
name = '유리'
height = 175.5
age = 21
hobby = 'programming'
print(name, '키는', height, ', 나이는', age, ', 취미는', hobby, '입니다!')
```

```
실행결과
유리 키는 175.5 , 나이는 21 , 취미는 programming 입니다!
```

한 칸씩 공백을 넣어 출력하는 콤마 때문에 띄어쓰기가 생겼지만, 변수 값만 나오는 것보다는 조금 더 문장이 매끄러워졌네요.

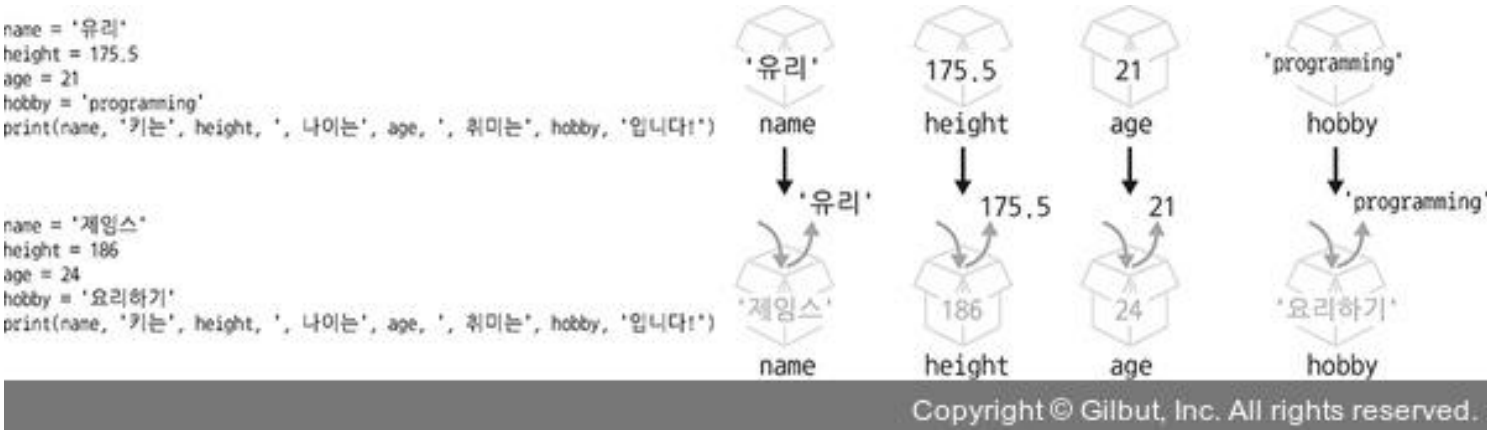
이번에는 앞의 코드를 그대로 넣고 그 아래에 새로운 값을 넣은 후 다시 한번 실행해 봅시다.

```
name = '유리'
height = 175.5
age = 21
hobby = 'programming'
print(name, '키는', height, ', 나이는', age, ', 취미는', hobby, '입니다!')

name = '제임스'
height = 186
age = 24
hobby = '요리하기'
print(name, '키는', height, ', 나이는', age, ', 취미는', hobby, '입니다!')
```

```
실행결과
유리 키는 175.5 , 나이는 21 , 취미는 programming 입니다!
제임스 키는 186 , 나이는 24 , 취미는 요리하기 입니다!
```

`print()` 명령어로 실행한 내용은 같은데, 첫 번째 출력과 두 번째 출력이 다르게 나옵니다. 이것은 첫 번째 출력 이후 4개의 변수에 다른 값이 저장되어서 그렇습니다. 이를 그림으로 표현하면 다음과 같습니다.



▲ 그림 3-3 변수 값의 변화

두 번째 그림에서 모든 변수에 새로운 값이 저장되면서 상자에서 값이 바뀌는 것이 보이나요? 변수에 새로운 값을 저장하면 이전 값은 사라진다는 것을 꼭 기억하세요!

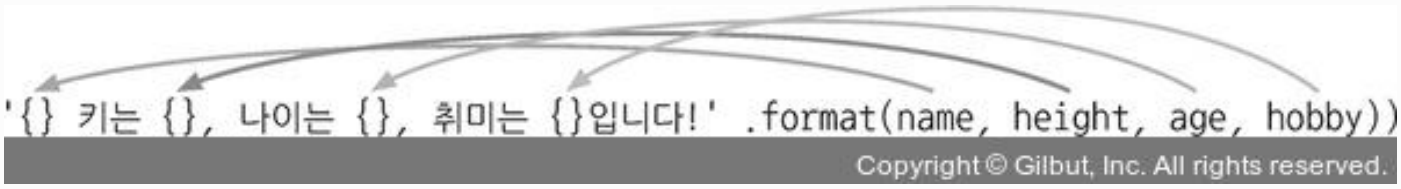
NOTE format() 명령어

한 문장에 변수와 문자를 구분하기 위해 콤마를 사용하는 대신 `.format()`을 사용해도 됩니다. 다음처럼 코드를 바꿔서 실행해 보세요.

```
name = '제임스'
height = 186
age = 24
hobby = '요리하기'
print(name, '키는', height, ', 나이는', age, ', 취미는', hobby, '입니다!')
print('{} 키는 {}, 나이는 {}, 취미는 {}입니다!' .format(name, height, age, hobby))
```

```
실행결과
제임스 키는 186 , 나이는 24 , 취미는 요리하기 입니다!
제임스 키는 186, 나이는 24, 취미는 요리하기입니다!
```

작은따옴표로 묶여진 한 문장 안에서 중괄호(`{}`) 부분이 변수에 저장된 값으로 변경되어 출력됩니다. `.format()` 명령어는 앞에 있는 문자들에서 `{}`를 찾아 `.format()`의 소괄호 안에 있는 값을 차례대로 하나씩 넣어 줍니다.



`.format()`은 파이썬에서 자주 활용합니다. 이 책에서는 `.format()`을 활용하지 않고도 충분히 진행할 수 있어서 사용하지 않지만, 직접 코드를 작성할 때는 한번 바꿔서 사용해 보세요.

변수에 연산 결과 저장하기
이번에는 변수에 숫자를 저장해 앞 장에서 배운 연산과 접목해 보겠습니다.

```
p = 3.141592
r = 5
print('원의 넓이 =', p * r ** 2)
```

```
실행결과
원의 넓이 = 78.5398
```

수학을 좋아하는 사람은 무슨 식인지 바로 알 겁니다. 원의 넓이를 구하는 공식 πr^2이죠. 공식을 몰라도 전혀 상관없습니다. 우리는 변수만 이해하면 되니까요.

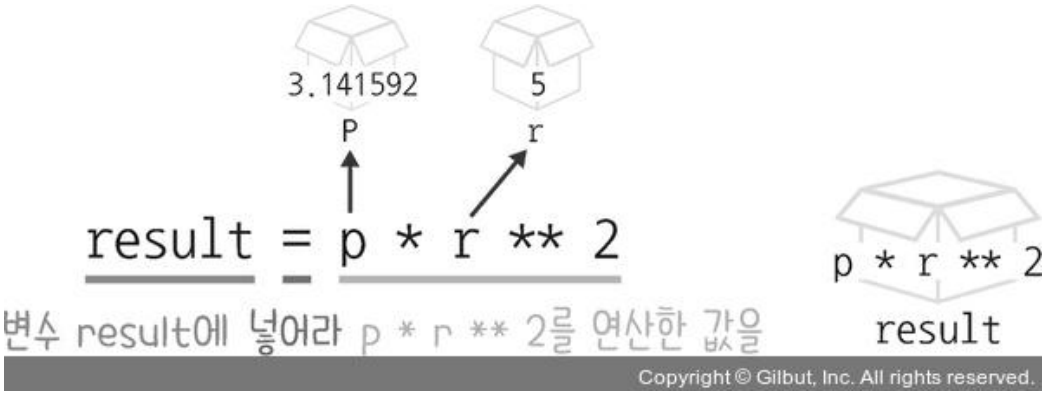
코드를 살펴보겠습니다. 먼저 변수 `p`에는 소수점 이하 여섯째 자리까지 표현한 원주율을, `r`에는 반지름 `5`를 넣었습니다. 그리고 파이썬의 연산 방식(`*`는 곱하기, `**`는 거듭제곱임을 잊지 않았죠?)으로 원의 넓이를 구하는 공식을 표현하고 결과를 출력합니다.

이번에는 앞의 코드를 다음과 같이 바꾸어 다시 실행합니다.

```
p = 3.141592
r = 5
result = p * r ** 2
print(result)
```

```
실행결과
78.5398
```

이번에는 원의 넓이를 계산한 결과를 새로운 변수 `result`에 넣었습니다. 이전에는 변수에 정해진 값을 저장하는 것만 배웠습니다. 앞의 코드를 보니 `result = p * r ** 2`처럼 변수에 연산 결과를 저장할 수도 있습니다. 코드를 조금 더 자세히 살펴보겠습니다.



▲ 그림 3-4 변수에 연산 결과 저장하기

이 코드도 역시 '`result` 변수에 `p * r ** 2`를 연산한 값을 넣어라'로 이해하면 됩니다. 코드를 실행하면 컴퓨터 내부에서는 메모리 공간을 찾아 `result`라 이름 붙이고 `p * r ** 2`를 연산한 결과를 저장합니다. 이때 `p`에는 `3.141592`가, `r`에는 `5`가 들어 있으니 결국 `3.141592 * 5 ** 2`를 연산해 `result`에 저장합니다.

3.1.3 변수를 사용하는 이유

다음 두 코드는 결과는 같지만, 오른쪽 코드가 한 줄 더 깁니다. 오른쪽 코드에서는 `p * r ** 2`의 결과를 다시 `result` 변수에 저장하지만, 왼쪽 코드에서는 결괏값을 변수에 저장하지 않기 때문입니다.

```
p = 3.141592
r = 5
print(p * r ** 2)
print('반지름이', r, '인 원의 넓이는', p * r ** 2)
```

```
실행결과
78.5398
반지름이 5 인 원의 넓이는 78.5398
```

```
p = 3.141592
r = 5
result = p * r ** 2
print(result)
print('반지름이', r, '인 원의 넓이는', result)
```

```
실행결과
78.5398
반지름이 5 인 원의 넓이는 78.5398
```

변수를 만들어 값을 저장하면 코드도 길어지고 번거로워 보입니다. 그런데도 변수를 사용하는 이유는 뭘까요?

첫 번째 이유는 기억하기 힘든 긴 데이터를 간단하게 저장할 수 있기 때문입니다. 원주율은 소수점 아래로 무한한 값이 있습니다. 이 많은 숫자를 기억하기가 어려워서 보통 3.14라고 줄여서 표현합니다. 변수를 활용하면 원주율처럼 긴 데이터를 계산할 때마다 기억하지 않아도 됩니다. 그저 변수 `p`에 저장한 긴 원주율 숫자를 불러오기만 하면 됩니다. 원의 넓이뿐만 아니라 원의 둘레, 반지름이 같은 구의 겉넓이도 변수 `p`에 저장된 값으로 구할 수 있습니다.

다음 코드를 실행해 보면 좀 더 이해하기 쉬울 겁니다. 참고로 원의 둘레를 구하는 공식은 $2\pi r$, 구의 겉넓이를 구하는 공식은 $4\pi r^2$입니다.

```
p = 3.141592
r = 5

# 원의 넓이
result1 = p * r ** 2
print('원의 넓이 =', result1)
# 원의 둘레
result2 = 2 * p * r
print('원의 둘레 =', result2)
# 구의 겉넓이
result3 = 4 * p * r ** 2
print('구의 겉넓이 =', result3)
```

```
실행결과
원의 넓이 = 78.5398
원의 둘레 = 31.41592
구의 겉넓이 = 314.1592
```

NOTE 주석

파이썬에서 코드 앞에 해시 기호(`#`)를 붙이면 해당 줄은 제외하고 실행합니다. 코드 실행에 전혀 영향을 주지 않는 부분으로, 이를 주석(comment)이라고 합니다. 코드가 어떤 역할이나 기능을 하는지 주석으로 설명을 넣을 수 있습니다. `#`은 한 줄짜리 주석을 처리할 때 사용합니다. 하지만 복잡한 프로그램을 다른 사람과 공유하려면 프로그램에 대한 설명을 상세하게 적어야 합니다. 이런 경우에 주석을 여러 줄로 작성해야 할 수 있습니다. 이때는 주석으로 처리하고 싶은 부분을 큰따옴표 3개(`"""`)로 감싸면 됩니다.

```
"""
원의 넓이를 구하는 프로그램입니다.
변수 p는 원주율을 저장한 변수이고,
변수 r는 반지름을 저장한 변수입니다.
"""
p = 3.141592
r = 5

# 원의 넓이
result1 = p * r ** 2
print('원의 넓이 =', result1)
```

변수는 긴 데이터를 저장할 때도 편하지만, 이것보다 더 강력한 장점이 있습니다. 변수를 사용하면 값을 재사용할 수 있습니다. 예를 들어, 앞의 코드에서 반지름을 5에서 17로 변경한다고 해 봅시다. 반지름 변수를 사용하지 않는다면 어떨까요? 코드로 비교해 봅시다.

```
# 반지름 변수를 사용하지 않을 때
p = 3.141592

result1 = p * 17 ** 2
print('원의 넓이 =', result1)

result2 = 2 * p * 17
print('원의 둘레 =', result2)

result3 = 4 * p * 17 ** 2
print('구의 겉넓이 =', result3)
```

```
# 반지름 변수를 사용할 때
p = 3.141592
r = 17

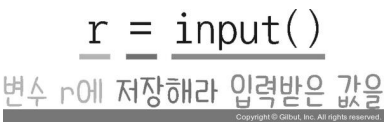
result1 = p * r ** 2
print('원의 넓이 =', result1)

result2 = 2 * p * r
print('원의 둘레 =', result2)

result3 = 4 * p * r ** 2
print('구의 겉넓이 =', result3)
```

기존 코드에서 반지름의 값 5를 17로 바꿔야 하는데, 왼쪽 코드처럼 반지름 변수를 사용하지 않으면 반지름이 필요한 모든 부분을 바꿔야 합니다. 총 3번 바꾸네요. 하지만 변수를 사용하면 변수에 값을 저장하는 처음 한 번만 바꾸면 됩니다. 훨씬 간단하죠. '3번만 바꾸면 되는데 뭐가 번거롭지?'라고 생각할 수 있습니다. 프로그램에 조금 더 요소를 넣어 보겠습니다.

이번에는 반지름을 직접 입력받아 원의 넓이와 둘레, 구의 겉넓이를 구하는 프로그램을 작성합니다. 오른쪽 코드에서 딱 한 줄만 변경하면 됩니다. 어디를 변경하면 될까요? 반지름을 바꾼다고 했으니 반지름 변수에 값을 저장하는 `r = 17`을 변경하면 되겠네요. 그리고 입력을 받아야 하니 2장에서 배웠던 입력 명령어 `input()`을 사용해야죠. 그럼 바꿔 봅시다. 앞서 배운 형식을 그대로 적용하면 됩니다.



이 코드를 해석하면 '변수 `r`에 입력받은 값을 저장하라'고 할 수 있습니다. 이대로 두면 어떤 값을 입력해야 하는지 알기 어렵겠죠? 여기에 안내문을 넣어 봅시다. 간단하게 '반지름은?'이라고 넣겠습니다. 추가로 결괏값 출력 부분은 한 줄로 작성합니다.

```
p = 3.141592
r = input('반지름은? ')

result1 = p * r ** 2
result2 = 2 * p * r
result3 = 4 * p * r ** 2

print('원의 넓이 =', result1, '원의 둘레 =', result2, '구의 겉넓이 =', result3)
```

```
실행결과
반지름은? 999
---------------------------------------------------------------------------
----> 4 result1 = p * r ** 2
TypeError: unsupported operand type(s) for ** or pow(): 'str' and 'int'
```

반지름을 입력하니 오류가 발생하네요. 오류는 나중에 설명하고, 우선 코드를 다음과 같이 조금만 수정합니다.

```
p = 3.141592
r = int(input('반지름은? '))

result1 = p * r ** 2
result2 = 2 * p * r
result3 = 4 * p * r ** 2

print('원의 넓이 =', result1, '원의 둘레 =', result2, '구의 겉넓이 =', result3)
```

```
실행결과
반지름은? 999
원의 넓이 = 3135311.9575920003 원의 둘레 6276.900816 구의 겉넓이 = 12541247.830368001
```

이번에는 결괏값이 제대로 나옵니다. 이처럼 복잡한 값을 저장하거나 값이 수시로 변경될 때 변수를 사용하는 것이 효율적입니다. 재실행하며 반지름에 다양한 숫자를 넣어 보세요!

3.1.4 변수를 사용할 때 유의할 점

변수를 사용할 때 지키면 좋은 규칙이 있습니다. 바로 의미 있는 이름으로 변수를 만드는 것입니다. 앞의 코드에서는 변수명을 `result1`, `result2`, `result3`으로 했는데 변수만 보면 어떤 값을 담고 있는지 바로 알기가 어렵습니다. 그래서 변수명은 변수에 저장하는 값이 의미하는 바를 나타내도록 정하는 것이 좋습니다. 그러면 프로그래밍하기도 편하고, 다른 사람들이 여러분의 코드를 볼 때도 이해하기가 훨씬 쉽습니다. 우리가 다른 사람의 코드를 볼 때도 편하고요.

그럼 앞의 코드를 바꿔 보겠습니다. 원의 넓이는 circle area를 줄여 `c_area`, 원의 둘레는 circle length를 줄여 `c_len`, 구의 넓이는 sphere area를 줄여 `s_area`라고 합시다.

```
p = 3.141592
r = int(input('반지름은? '))

c_area = p * r ** 2
c_len = 2 * p * r
s_area = 4 * p * r ** 2

print('원의 넓이 =', c_area, '원의 둘레 =', c_len, '구의 겉넓이 =', s_area)
```

```
실행결과
반지름은? 999
원의 넓이 = 3135311.9575920003 원의 둘레 = 6276.900816 구의 겉넓이 = 12541247.830368001
```

어떤 코드인지 파악하기가 좀 더 쉬워졌습니다. 사소한 차이일 수 있지만, 나중에 더 길고 정교한 프로그램을 만들게 되면 이런 직관적인 변수명이 더욱 중요해집니다.

3.1.5 변수를 사용해 간단한 프로그램 만들기 1

지금까지 배운 내용을 바탕으로 간단한 프로그램을 만들어 보겠습니다.

1. 자기 소개 프로그램을 작성해 봅시다. 이름, 나이, 특징을 입력받아 변수에 저장한 후 다음처럼 출력합니다.

실행결과

입력받는 부분

```
이름 : 고라니
나이 : 4
특징 : 애교가 많은 편
```

입출력하는 부분

```
고라니(4) 애교가 많은 편
애교가 많은 편인 고라니
```

힌트 프로그램을 작성하기 전에 변수가 몇 개 필요한지, 변수명은 어떻게 지을지 생각해 보세요! 그러고 나서 생각한 것과 결과가 같은지 비교하세요.

앞에서 모두 배운 내용이라 어렵지 않습니다. 다만 여러 요소를 연결해 출력하는 콤마와 + 연산자의 차이점을 잘 파악하고 적절히 활용할 수 있어야 합니다. 먼저 '고라니(4)'와 '애교가 많은 편인'을 출력할 때는 공백 없이 문자를 연결해야 하므로 + 연산자를 사용합니다. ')' 뒤에 오는 '애교가 많은 편'과 '인' 뒤에 오는 '고라니'를 출력하려면 ')' 뒤와 '인' 뒤에 공백을 한 칸 만든 후 특징을 출력해야 하므로 콤마를 사용합니다. 코드를 바꾸어 가며 출력 형태를 더 다양하게 만들어 연습해 보세요.

```
name = input('이름 : ')
age = input('나이 : ')
feature = input('특징 : ')
print(name + '(' + age + ')', feature)
print(feature + '인', name)
```

NOTE 자동 완성 기능

에디터를 사용할 때 '자동 완성' 기능을 활용하면 만들어 둔 변수나 미리 만들어진 명령어를 일일이 작성하지 않아도 돼서 코딩 속도를 빠르게 할 수 있습니다.

코랩에서는 [설정 → 편집기 → 코드 완성 제안을 자동으로 표시]에 체크되어 있으면 별다른 키를 누르지 않아도 자동으로 나타납니다(설정 메뉴는 코랩 파일의 오른쪽 상단에 있는 톱니바퀴 모양 버튼을 클릭하면 나옵니다). 이후 명령어나 변수의 첫 글자를 입력하면 작은 창이 뜨면서 클릭할 수 있는 요소가 몇 개 나옵니다. 마우스로 클릭하거나 위아래 화살표로 이동해 Enter를 누르면 자동으로 작성됩니다. 다만, 변수는 정의한 후부터 자동 완성 기능이 적용됩니다. 주피터 노트북에서는 첫 글자 다음에 Tab키를 눌러 줘야 합니다. 이후 선택 방법은 같습니다.



▲ 그림 3-5 자동 완성 기능

자동 완성 기능은 코드를 잘못 작성하는 것을 막고 프로그래밍을 편하게 작성하도록 도와줍니다. 자동 완성 기능이 있으니 좀 길더라도 의미 있는 변수명을 만들어 보세요.

2. 1년마다 키가 10cm씩 자라는 농구선수가 있습니다. 2017년에는 175.5cm, 2018년에는 185.5cm, 2019년에는 195.5cm, 2020년에는 205.5cm였습니다. 연도별 농구선수의 키를 다음처럼 출력해 봅시다.

실행결과
```
2017 년 - 175.5 cm
2018 년 - 185.5 cm
2019 년 - 195.5 cm
2020 년 - 205.5 cm
```

힌트 프로그램을 작성하기 전에 실행결과를 보고 변하는 부분과 변하지 않는 부분을 나눠 보면 어디를 변수로 만들어야 할지 알 수 있습니다.

먼저 연도가 2017, 2018, 2019, 2020으로 1씩 증가하며 변하는 것이 보입니다. 그다음 키가 175.5, 185.5, 195.5, 205.5로 10씩 증가합니다. 변하는 부분이 두 군데이니 변수를 2개 만들어야 합니다. 연도를 저장하는 변수를 `year`로, 키를 저장하는 변수를 `height`로 정하고 일단 첫 줄만 출력하게 작성해 보겠습니다.

```
year = 2017
height = 175.5
print(year, '년 -', height, 'cm')
```

실행결과
```
2017 년 - 175.5 cm
```

NOTE 콤마(,)와 + 연산자

오류가 발생한다면 콤마 대신에 + 연산자를 사용하지 않았는지 살펴보세요.

```
year = 2017
height = 175.5
print(year + '년 -' + height + 'cm')
```

실행결과

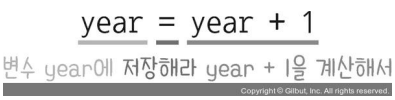
```
----> 3 print(year + '년 -' + height + 'cm')
TypeError: unsupported operand type(s) for +: 'int' and 'str'
```

공백이 생기지 않게 `print(year + '년 -' + height + 'cm')`로 작성하면 오류가 발생합니다. 결과창에 어떤 오류인지 나타납니다. 오류를 해석하는 방법은 다음 절에 배우므로 여기에서는 생략하겠습니다. 하지만 오류를 해석하지 않아도 이미 배운 부분이라서 쉽게 알 수 있습니다. + 연산자는 '숫자 + 숫자'나 '문자 + 문자'일 때는 문제가 없습니다. 그러나 '문자 + 숫자'처럼 다른 요소를 +로 연결하면 오류가 발생합니다. 앞에서도 설명했듯이 `year`는 양옆에 작은따옴표가 없으니 문자가 아닌 변수로 인식합니다. 숫자와 문자를 +로 연결해서 오류가 발생한 겁입니다. 공백이 신경 쓰이더라도 일단은 콤마를 사용하세요.

다음으로 `year`에 2018을 넣습니다. `year = 2018`처럼 `year`에 바로 2018이라는 값을 넣을 수도 있지만, 연도가 1씩 증가하므로 연산자를 활용해 봅시다. 새로운 `year`에는 기존 `year`에서 1이 증가한 값이 들어가야 합니다. 그러면 다음과 같이 표현할 수 있습니다.

```
year = year + 1
```

이 코드는 '변수 `year`에 `year + 1`을 계산한 값을 넣어라'로 해석할 수 있습니다. 이때 변수 `year`에는 숫자가 저장되어 있으므로 `year + 1`은 숫자 + 숫자가 되어 계산이 가능합니다.



현재 `year`의 값이 2017이므로 1을 더한 값인 2018을 `year`에 다시 저장한다는 의미죠. 그러면 `year`에서는 2017이 사라지고 2018이 새로 저장됩니다. 이와 마찬가지로 키를 저장하는 변수 `height` 값을 10씩 증가시키는 코드도 작성할 수 있습니다.

```
year = 2017
height = 175.5
print(year, '년 -', height, 'cm')
year = year + 1
height = height + 10
print(year, '년 -', height, 'cm')
```

```
실행결과
2017 년 - 175.5 cm
2018 년 - 185.5 cm
```

`print()` 명령어에 사용한 변수는 같은데, 변수 값이 바뀌어 결과도 다른 값이 나옵니다. 남은 2019년과 2020년도 추가해 봅시다.

```
year = 2017
height = 175.5
print(year, '년 -', height, 'cm')
year = year + 1
height = height + 10
print(year, '년 -', height, 'cm')
year = year + 1
height = height + 10
print(year, '년 -', height, 'cm')
year = year + 1
height = height + 10
print(year, '년 -', height, 'cm')
```

실행결과

```
2017 년 - 175.5 cm
2018 년 - 185.5 cm
2019 년 - 195.5 cm
2020 년 - 205.5 cm
```

변수에 저장된 값이 어떻게 변하는지 직접 확인하면서 프로그램을 작성해 보세요!

하나 더, 여기서 코드양을 줄일 방법이 있습니다. `year = year + 1`, `height = height + 10`은 현재 변수의 값에 숫자를 더해 다시 해당 변수에 넣는다는 공통점이 있습니다. 그런데 등호를 기준으로 왼쪽과 오른쪽에 변수명이 반복됩니다. 그래서 다음과 같이 줄여 씁니다.

```
year += 1
height += 10
```

앞의 코드를 수정하면 다음과 같습니다.

```
year = 2017
height = 175.5
print(year, '년 -', height, 'cm')
year += 1
height += 10
print(year, '년 -', height, 'cm')
year += 1
height += 10
print(year, '년 -', height, 'cm')
year += 1
height += 10
print(year, '년 -', height, 'cm')
```

실행하면 같은 결과가 나옵니다. 앞으로 이런 형태의 코드가 나오면 이 방법을 활용해 코드양을 줄여 보세요.

NOTE 연산자 줄이기

앞에서 보듯이 `a = a + 1`은 `a += 1`로 줄여 쓸 수 있습니다. 그럼 다른 연산자는 어떨까요? 코드를 실행해 보면 알겠지만, 다른 연산자도 줄여 쓸 수 있습니다.

▼ 표 3-1 연산자 줄이기

줄이기 전	줄인 후
`a = a + 5`	`a += 5`
`a = a - 9`	`a -= 9`
`a = a * 7`	`a *= 7`
`a = a / 2`	`a /= 2`

1분 퀴즈

1 변수에 대한 다음 설명 중 틀린 것을 고르세요.

① `x = 99`가 실행되면 `x`라는 변수에 `99`가 저장된다.

② 변수명은 변수에 담길 값의 특징을 나타내는 것이 좋다.

③ 값이 자주 바뀔 때 변수를 사용하면 프로그램 수정을 최소화할 수 있다.

④ `a` 변수에 `22`가 저장되어 있을 때 `print('a')`를 실행하면 `22`가 출력된다.

⑤ `s = s * 90`은 `s *= 90`으로 줄여 쓸 수 있다.

정답 및 해설

3.2

변수에 저장되는 데이터의 유형: 데이터 타입

3.2.1 데이터 타입이란

변수의 의미와 특징을 알아보면서 변수에 숫자, 문자 등 다양한 값을 저장했습니다. 이렇게 변수에 저장한 숫자나 문자 등을 모두 데이터라고 합니다.

앞에서 작성한 코드 중 일부를 다시 살펴볼까요?

```
year = 2016
height = 175.5
name = '제임스'
```

코드를 실행하면 `year`와 `height` 변수에는 `2016`과 `175.5`라는 숫자 데이터가, `name` 변수에는 `'제임스'`라는 문자 데이터가 저장됩니다. 숫자와 문자는 다른 유형의 데이터이기 때문에 이를 구분하는 방법으로 코드에서 문자 데이터 앞뒤에만 작은따옴표를 넣습니다.

그런데 2016과 175.5도 같은 숫자지만, 생김새가 조금 다릅니다. 2016은 소수점이 없고, 175.5에는 소수점이 있죠. 이렇게 숫자에 소수점이 없으면 정수, 소수점이 있으면 실수라고 합니다. 따라서 2016은 정수 데이터, 175.5는 실수 데이터입니다.

프로그래밍에서는 이런 정수, 실수, 문자를 데이터 타입(data type)이라고 합니다. 데이터 타입은 데이터(data)와 타입(type) 두 단어를 합친 말로, 데이터는 자료를 의미하고, 타입은 유형이나 형태를 의미합니다. 그러므로 데이터 타입은 자료의 유형이라고 할 수 있죠. 그래서 데이터 타입을 자료형이라고도 합니다.

앞에 나온 내용을 조금 바꿔서 말하면 2016은 정수이므로 데이터 타입이 정수형입니다. 그래서 2016이라는 정수 데이터를 담고 있는 `year` 변수는 자동으로 데이터 타입이 정수형이 됩니다. 코드로 확인해 볼까요?

```
year = 2016
print(type(year))
```

```
실행결과
<class 'int'>
```

`type()`은 데이터 타입을 알려 주는 명령어로, 소괄호 안에 데이터 타입을 알고 싶은 값을 넣습니다. `type(year)`의 결과를 눈으로 확인하기 위해 `print()`도 추가했습니다.

실행결과의 `class`는 일단 '그룹' 정도로 이해하면 됩니다. 나중에 설명하니 여기서는 크게 신경쓰지 않아도 됩니다. `year`의 데이터 타입이 `int`라고 나왔네요. `int`는 정수를 뜻하는 integer의 약자로, 정수형을 뜻합니다. 마찬가지로 175.5는 실수 데이터이므로 175.5를 담고 있는 변수 `height` 역시 자동으로 데이터 타입이 실수형이 됩니다.

```
height = 175.5
print(type(height))
```

```
실행결과
<class 'float'>
```

이번에는 `float`가 나오네요. 부동소수점을 의미하는 floating point를 줄여 `float`라고 합니다. floating point는 직역하면 '떠다니는 점'인데, 실수에 등장하는 소수점이 숫자들 사이에서 떠다니는 듯 보여서 이렇게 부른다고 합니다.

'제임스'로 넘어가 봅시다. '제임스'는 문자입니다. 그러니 '제임스'가 들어 있는 `name` 변수의 데이터 타입도 당연히 문자형이 됩니다.

```
name = '제임스'
print(type(name))
```

```
실행결과
<class 'str'>
```

파이썬에서 문자는 string의 약자인 `str`로 표현합니다. string은 원래 끈, 줄 등을 의미하는데, 컴퓨터에서는 하나 이상의 문자나 단어를 의미하게 됐습니다. 그래서 문자 데이터의 데이터 타입은 문자열(string)이라고 합니다.

그럼 지금까지 나온 데이터 타입을 정리해 봅시다.

▼ 표 3-2 파이썬의 데이터 타입

데이터 타입		파이썬에서 데이터 타입 표기	의미
숫자형	정수형	`int`	integer
	실수형	`float`	floating point
문자열		`str`	string

3.2.2 초보자가 겪는 흔한 오류

2장 55쪽에 나온 나온 코드를 다시 한번 봅시다. `TypeError`가 발생한 코드입니다.

```
print('123' + 2)
```

```
실행결과
----> 1 print('123' + 2)
TypeError: must be str, not int
```

이렇게 오류가 발생하면 오류 내용의 가장 마지막 줄을 먼저 보는 게 좋습니다. 오류의 종류와 구체적 내용을 보여 주거든요. 마지막 줄에 `TypeError: must be str, not int`라고 나옵니다(주피터 노트북 등의 일부 에디터에서는 `TypeError: can only concatenate str (not "int") to str`로 나옵니다. 의미는 동일하니 당황하지 마세요).

프로그래밍 언어들은 외국에서 만들어지다 보니 설명이 대부분 영어로 나옵니다. 수준 높은 독해력이 필요하지 않으니 모르는 단어는 인터넷으로 찾아가며 직접 해석해 보기를 권합니다. 오류 내용을 해석해 보면 “타입오류: 정수형이 아닌 문자열이어야 한다”가 되겠네요. 문자와 숫자를 더해서 오류가 발생한 것이죠. 데이터 타입을 공부하고 나니 오류 내용이 이제 이해됩니다.

'3.1.3 변수를 사용하는 이유'에 나온 원의 넓이와 둘레, 구의 겉넓이를 구하는 코드도 다시 한번 살펴볼까요? 반지름을 입력받는 부분에서 오류가 났습니다. 오류 내용을 직접 해석해 봅시다.

```
p = 3.141592
r = input('반지름은? ')

result1 = p * r ** 2
result2 = 2 * p * r
result3 = 4 * p * r ** 2

print('원의 넓이 =', result1, '원의 둘레 =', result2, '구의 겉넓이 =', result3)
```

```
실행결과
반지름은? 999
---------------------------------------------------------------------------
----> 4 result1 = p * r ** 2
TypeError: unsupported operand type(s) for ** or pow(): 'str' and 'int'
```

4번째 줄에서 오류가 발생했습니다. 마지막 줄의 오류 내용을 해석해 보면 “타입오류: `**` 또는 거듭제곱에서 지원하지 않는 연산자 타입: ‘문자열’과 ‘정수형’”입니다. 문자와 숫자 간에는 제곱 연산이 불가능하다는 뜻입니다. 그럼 `p * r ** 2` 중 어떤 변수가 `str`(문자열)일까요? 변수 `p`는 프로그램 첫 줄에서 실수 형태의 원주율을 저장했기 때문에 `float`입니다. 그러면 `r`이 `str`이겠네요. 그런데 조금 이상하죠. 입력 값으로 999를 넣었으니 `int`(정수) 타입이어야 하는데 왜 `str`일까요? 다음 코드로 확인해 봅시다.

```
a = input()
print(type(a))
```

입력칸에 '안녕하세요'를 입력하면 어떤 데이터 타입이 나올까요?

```
실행결과
안녕하세요
<class 'str'>
```

`str`이 나옵니다. 문자를 넣었으니 당연히 `str`이 나오죠. 그럼 999를 입력했을 때 결과를 예상해 보세요. `int`가 나올까요?

```
실행결과
999
<class 'str'>
```

당연히 `int`가 나올 줄 알았는데 `str`이 나왔네요. 어떻게 된 걸까요? 그 비밀은 `input()`에 숨어 있습니다. `input()`은 데이터를 입력받는 명령어입니다. 그런데 파이썬에서는 데이터가 문자든, 숫자든 상관없이 `input()`으로 입력받는 모든 값을 그냥 문자열로 취급합니다. 그래서 앞에서 `TypeError`가 발생한 것이죠.

> `input()`으로 입력받는 데이터는 모두 문자열(str)로 인식한다

따라서 입력받은 데이터로 계산하려면 문자열을 숫자형으로 바꿔야 합니다. 이처럼 프로그램에서는 종종 변수의 데이터 타입을 바꿔야 하는 일이 생깁니다. 문자열을 숫자형으로 또는 숫자형을 문자열로 바꾸기도 하죠. 그럼 데이터 타입은 어떻게 바꿀까요?

3.2.3 데이터 타입 바꾸기

변수의 데이터 타입을 바꾸는 방법은 매우 간단합니다. 변경하려는 데이터 타입으로 해당 변수를 감싸면 됩니다. 만약 소수점까지 표현할 수 있는 실수로 바꾸고 싶다면 바꾸고 싶은 변수를 실수형을 나타내는 `float()`로 감싸 줍니다.

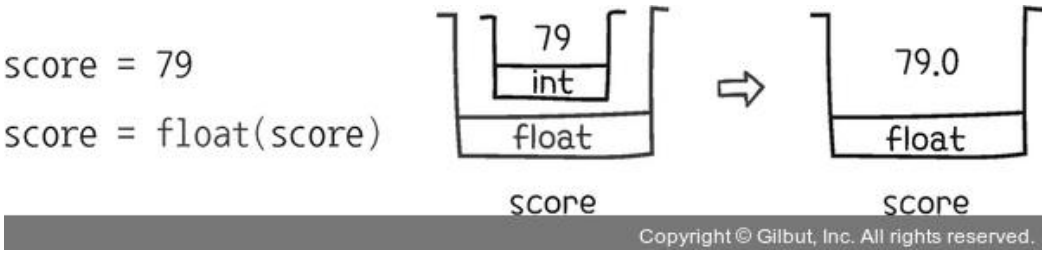
```
score = 79
print(type(score), score)

# 실수형으로 바꾸기
score = float(score)
print(type(score), score)
```

```
실행결과
<class 'int'> 79
<class 'float'> 79.0
```

`score` 변수의 데이터 타입은 `int`(정수형)로 나옵니다. 데이터 타입을 바꾸고 싶은 `score` 변수를 `float()`로 감싸 실수형으로 바꾸고 이를 다시 변수 `score`에 저장합니다. 그러고 나서 `score` 변수의 데이터 타입과 저장된 값을 출력합니다. `score`의 데이터 타입은 `float`고, 저장된 값은 실수로 변환한 79.0이 나옵니다.

이해하기 쉽게 그림으로 볼까요? 원래 79가 저장되어 있던 `score` 변수를 `float()`로 감싸 주면 실수형으로 값이 변경됩니다.



▲ 그림 3-6 정수형 데이터를 실수형으로 바꾸기

이번에는 실수형을 정수형으로 바꿔 볼까요? 어떻게 하면 될까요? 맞습니다. 바꾸려는 값이 저장된 변수를 정수형을 나타내는 `int()`로 감싸면 됩니다.

```
avg = 84.3
print(type(avg), avg)

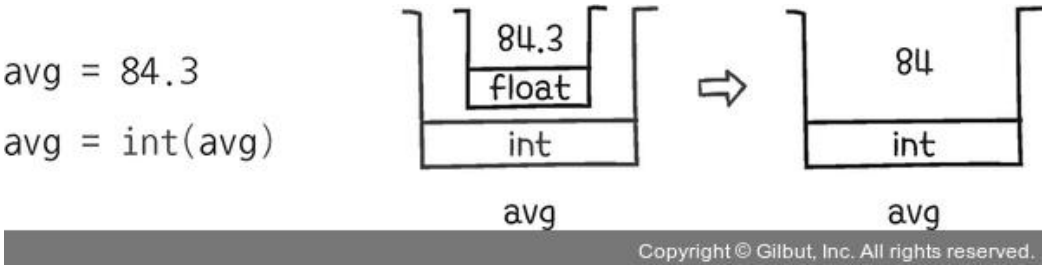
# 정수형으로 바꾸기
avg = int(avg)
print(type(avg), avg)
```

실행결과

```
<class 'float'> 84.3
<class 'int'> 84
```

TIP 가끔 `int(avg)`만 타이핑하고 실행하는 사람이 종종 있습니다. `int(avg)`와 `avg = int(avg)`는 매우 다릅니다. 데이터 타입을 바꾸는 것은 동일하지만, 바꾼 값을 변수에 다시 저장하느냐 저장하지 않느냐의 차이가 있습니다. 데이터 타입을 바꾸고 다시 변수에 저장하지 않으면 바꾼 내용을 컴퓨터가 기억하지 않으므로 의미가 없습니다. 데이터 타입을 바꾼 후 다시 저장하기를 잊지 마세요.

84.3이 저장된 `avg` 변수를 `int()`로 감싸서 정수형으로 만들었습니다. 이때 유의해야 할 부분이 있습니다. 실수형을 정수형으로 바꾸는 것은 소수점이 있는 형태에서 없는 형태로 변경하는 것이어서 소수점과 소수점 이하의 수를 버리게 됩니다. `int`로 데이터 타입을 변경할 때는 이 부분에 유의하세요!



▲ 그림 3-7 실수형 데이터를 정수형으로 바꾸기

그러면 정수형이나 실수형을 문자열로 바꾸고 싶을 때는 어떻게 할까요? 이도 마찬가지로 변경하려는 변수를 문자열을 나타내는 `str()`로 감싸면 됩니다.

```
x = 32
y = 32.9

x = str(x)
y = str(y)

print(type(x), type(y))
print(x + y)
```

실행결과

```
<class 'str'><class 'str'>
3232.9
```

정수 `32`가 들어 있는 `x` 변수를 `str()`로 감싸면 데이터가 문자 `'32'`로 바뀌어 다시 `x`에 저장됩니다. `y` 변수 역시 문자 `'32.9'`로 변경되어 저장됩니다. 두 변수가 모두 문자로 변환됐기 때문에 `print(x + y)`를 실행했을 때 숫자 + 숫자가 아닌 문자 + 문자 연산을 합니다. 그래서 64.9가 아닌 3232.9가 출력됩니다.

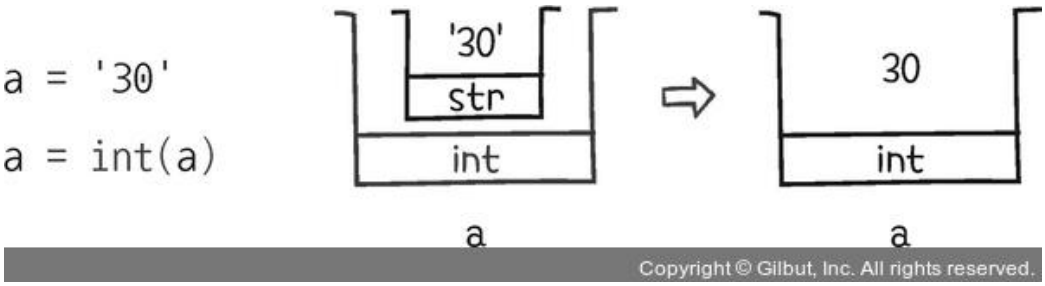
이번에는 문자열을 숫자형으로 변경해 볼까요?

```
a = '30'
print(type(a))

a = int(a)
print(type(a), a * 3)
```

```
실행결과
<class 'str'>
<class 'int'> 90
```

문자열인 `a` 변수를 `int()`로 감싸서 숫자형으로 만듭니다. 마지막 줄에서 `a * 3`을 하니 숫자 연산이 되어 출력됩니다. 만약 `a`가 숫자형으로 바뀌지 않았다면 90이 아니라 303030이 출력됐겠죠?



▲ 그림 3-8 문자열 데이터를 정수형으로 바꾸기

그런데 숫자를 작은따옴표로 감싼 문자열이 아니라 진짜 문자열을 숫자형으로 바꾸면 어떻게 될까요? 예를 들어, `a` 변수에 `'제임스'`가 저장됐을 때 `int()`로 감싸면 정수형으로 바뀔까요? 한번 실행해 봅시다.

```
a = '제임스'
print(type(a))

a = int(a)
print(type(a), a * 3)
```

```
실행결과
<class 'str'>
---------------------------------------------------------------------------
----> 4 a = int(a)
ValueError: invalid literal for int() with base 10: '제임스'
```

역시 오류가 발생합니다. 4번째 줄에서 오류가 발생했네요. 마지막 줄의 `ValueError` 내용을 해석해 봅시다. base 10은 '10진법'을 의미하므로 "값오류: 10진법에서 정수형에 유효하지 않은 문자: '제임스'"라고 해석할 수 있습니다. 이는 `a` 변수에 저장된 '제임스'를 정수형으로 바꿀 수 없다는 뜻입니다. 한글이나 알파벳으로 된 문자는 `int()`로 감싸도 정수형으로 바꿀 수 없습니다. `float()`도 마찬가지고요.

> 제제쌤의 조언
> 파이썬은 데이터 타입만 잘 이해해도 훨씬 수월하게 프로그래밍할 수 있습니다. 데이터 타입이 다소 지루하게 느껴지더라도 천천히 꼼꼼하게 학습해서 자신의 것으로 만들길 바랍니다.

그러면 3.2.2 초보자가 겪는 흔한 오류에서 본 두 가지 오류를 고치려면 어떻게 해야 할까요?

첫 번째 오류는 문자와 숫자 사이에 +를 사용할 수 없는 오류였죠. 해결 방법은 두 가지입니다. 문자를 숫자로 바꾸거나 숫자를 문자로 바꾸는 방법입니다.

수정 전 코드

```
print('123' + 2)
```

수정 후 코드

```
print('123' + '2')
# 또는
print(int('123') + 2)
```

> 실행결과
> 1232
> 125

`input()`으로 반지름을 입력받은 101쪽 두 번째 코드의 오류를 봅시다. 앞에서 오류를 해결할 때 `input('반지름은? ')`을 `int()`로 감싸서 실행했습니다.

```
p = 3.141592
r = int(input('반지름은? '))
# 또는 두 줄로
# r = input('반지름은? ')
# r = int(r)

result1 = p * r ** 2
result2 = 2 * p * r
result3 = 4 * p * r ** 2

print('원의 넓이 =', result1, '원의 둘레 =', result2, '구의 겉넓이 =', result3)
```

```
실행결과
반지름은? 999
원의 넓이 = 3135311.9575920003 원의 둘레 6276.900816 구의 겉넓이 = 12541247.830368001
```

`input()`으로 입력받으면 값이 문자열로 인식됩니다. 그러므로 입력받은 값을 정수형으로 바꿔야 합니다. `input('반지름은?')` 부분을 `int()`로 감싸면 문자열로 인식된 입력값이 정수형으로 다시 바뀌어 변수 `r`에 저장됩니다.

```
r = input('반지름은? ') → str
            ↓
r = int(input('반지름은? ')) → int
```

3.2.4 변수를 사용해 간단한 프로그램 만들기 2

지금까지 내용을 바탕으로 간단한 프로그램을 만들어 보겠습니다.

1. 마트에서 과자와 라면, 햄을 사려고 합니다. 과자와 라면, 햄의 개수를 각각 입력받아 총 금액이 얼마인지 알려 주는 프로그램을 작성해 봅시다. 이때 과자는 1200원, 라면은 800원, 햄은 2400원이고, 다음처럼 출력합니다.

실행결과

입력받는 부분

```
과자를 몇 개 구매하겠습니까? 3
라면을 몇 개 구매하겠습니까? 5
햄을 몇 개 구매하겠습니까? 2
```

출력하는 부분

```
총 금액: 12400
```

힌트 이 프로그램에는 과자의 가격과 개수, 라면의 가격과 개수, 햄의 가격과 개수, 총 금액까지 모두 7개의 요소가 있습니다. 이 모든 요소를 변수로 만들어야 할까요? 프로그램 안에서 변하는 요소와 변하지 않는 요소를 잘 구분해서 작성해 보세요.

7가지 요소 중 변하는 부분은 입력받는 부분인 과자 개수, 라면 개수, 햄 개수일 겁니다. 따라서 이 세 요소를 변수에 담는 것이 좋죠. 과자 개수는 snack count 줄여 `s_cnt`, 라면 개수는 ramen count를 줄여 `r_cnt`, 햄 개수는 ham count를 줄여 `h_cnt`로 변수명을 짓겠습니다. 그리고 총 금액도 입력받는 개수에 따라 변하니 변수에 저장하는 게 좋겠네요. `price`라고 하겠습니다. 이렇게 변수로 만들어 두면 나중에 필요할 때 변수명으로 언제든지 활용할 수 있습니다. 변수가 정해졌으니 이를 이용해 코드를 작성하면 됩니다.

```
s_cnt = int(input('과자를 몇 개 구매하겠습니까? '))
r_cnt = int(input('라면을 몇 개 구매하겠습니까? '))
h_cnt = int(input('햄을 몇 개 구매하겠습니까? '))

price = 1200 * s_cnt + 800 * r_cnt + 2400 * h_cnt
print('총 금액:', price)
```

2. 1번 문제의 마트에서 25% 할인 행사를 한다고 합니다. 앞의 코드를 조금 수정해 할인 전 금액과 할인 후 금액을 각각 출력해 봅시다.

`실행결과`

```
과자를 몇 개 구매하시겠습니까? 3
라면을 몇 개 구매하시겠습니까? 5
햄을 몇 개 구매하시겠습니까? 2
할인 전 금액: 12400 , 할인 후 금액: 9300.0
```

`변경된 부분`

총 금액을 `price`에 저장해 두었기 때문에 이를 활용하면 쉽게 완성할 수 있습니다.

```
s_cnt = int(input('과자를 몇 개 구매하겠습니까? '))
r_cnt = int(input('라면을 몇 개 구매하겠습니까? '))
h_cnt = int(input('햄을 몇 개 구매하겠습니까? '))

price = 1200 * s_cnt + 800 * r_cnt + 2400 * h_cnt
print('할인 전 금액:', price, ', 할인 후 금액:', price * 0.75)
```

NOTE 정수와 실수의 연산

결과에서 조금 이상한 점이 있죠. 할인 전 금액은 정수였는데, 할인 후 금액은 소수점이 붙은 실수로 나옵니다. 이는 컴퓨터가 숫자를 연산할 때 두 값 중 하나라도 실수면 실수를 기준으로 연산해서 그렇습니다. 정수보다 범위가 더 큰 실수를 기준으로 연산해야 버림하는 부분 없이 더 정확하게 계산 결과가 나오기 때문입니다. 확인해 볼까요?

```
print(9 + 2.0)
print(9 - 2.0)
print(9 * 2.0)
print(9 / 2.0)
```

```
실행결과
11.0
7.0
18.0
4.5
```

정수와 실수의 사칙연산 결과는 모두 실수로 나옵니다. 이를 정수 형태로 바꾸어 출력하고 싶다면 앞에서 배운 대로 `int()`로 감싸면 됩니다.

그런데 정수와 정수를 연산해도 결과가 실수로 나올 때가 있습니다. 바로 나눗셈입니다.

```
print(5 + 3)
print(5 - 3)
print(5 * 3)
print(5 / 3)
```

```
실행결과
8
2
15
1.6666666666666667
```

정수 5와 3을 사칙연산하면 나눗셈에서만 결과가 실수로 나옵니다. 나머지가 발생하지 않는 나눗셈도 실수로 나오므로 파이썬에서는 나눗셈을 하면 결과는 무조건 실수로 나온다는 점을 기억해 두세요.

3. 정수 2개를 입력받아 사칙연산하는 프로그램을 작성해 봅시다.

실행결과

입력받는 부분

```
숫자1 입력: 12
숫자2 입력: 4
```

출력하는 부분

```
두 수를 더한 값: 16
두 수를 뺀 값: 8
두 수를 곱한 값: 48
두 수를 나눈 값: 3.0
```

결과 코드를 보기 전에 먼저 스스로 코드를 작성해 보세요! 프로그래밍 실력 향상에 큰 도움이 됩니다!

힌트 숫자를 입력받으려면 뭐가 필요할까요? 입력받은 숫자는 어떻게 저장할까요? 변수는 몇 개가 필요할까요?

숫자 2개를 입력받으니 `input()`도 2개가 필요하겠죠? 또한, 숫자 2개를 각각 저장해야 하므로 변수도 2개 필요합니다. 변수명은 간단하게 `num1`, `num2`로 하겠습니다. 입력받는 숫자를 변수에 저장하는 코드는 쉽게 만들 수 있습니다. `input('숫자1 입력: ')`과 `input('숫자2 입력: ')`으로 입력받아 각각 `num1`, `num2`에 저장하면 됩니다. 그리고 `input()`으로 입력받은 모든 데이터는 문자열입니다. 숫자를 입력해도 문자열로 인식합니다. 따라서 `int()`로 감싸서 정수형으로 바꿔야 합니다. 이제 변수 `num1`, `num2`에 숫자가 저장됐으니 변수끼리 사칙연산한 후 안내문과 함께 `print()`로 출력하기만 하면 됩니다.

```
num1 = int(input('숫자1 입력: '))
num2 = int(input('숫자2 입력: '))

print('두 수를 더한 값:', num1 + num2)
print('두 수를 뺀 값:', num1 - num2)
print('두 수를 곱한 값:', num1 * num2)
print('두 수를 나눈 값:', num1 / num2)
```

1분 퀴즈

2 다음 중 오류가 발생하는 코드를 고르고, 오류가 발생하지 않는 나머지 4개 항목의 결괏값을 예상해 보세요.

①
```
a = 99
print(str(a) +'.9')
```

②
```
b = '45'
print(float(b) / 3)
```

③
```
c = input('숫자 입력: ')
print(c / 5)
```

④
```
d = input('숫자 입력: ')
print(d * 5)
```

⑤
```
e = int(input('숫자 입력: ')) # 단, 숫자형 데이터가 입력된다고 가정
print(e * 5)
```

정답 및 해설

3

마무리

이 장에서 배운 내용을 정리해 보겠습니다.

1 변수 데이터를 저장하는 공간으로 저장된 데이터는 변경될 수 있다. 변수명은 저장된 데이터의 특징을 잘 살린 이름으로 짓는 것이 좋다.

2 변수 만들기

```
# 변수명 = 값
a = 10
```

`a`라는 이름의 변수를 만들고 10을 저장한다.

3 변수에 값을 연산하는 식을 넣을 수도 있다.

```
# 변수명 = 연산식
result = p * r ** 2
```

`result`에 `p * r ** 2`를 연산한 값을 저장한다.

4 변수 값을 연산해 같은 변수에 다시 넣을 경우 다음처럼 줄여 쓸 수 있다.

줄이기 전	줄인 후	의미
`a = a + b`	`a += b`	`a`의 값에 `b`의 값을 더한 결괏값을 `a`에 다시 저장하기
`a = a - b`	`a -= b`	`a`의 값에 `b`의 값을 뺀 결괏값을 `a`에 다시 저장하기
`a = a * b`	`a *= b`	`a`의 값에 `b`의 값을 곱한 결괏값을 `a`에 다시 저장하기
`a = a / b`	`a /= b`	`a`의 값을 `b`의 값으로 나눈 결괏값을 `a`에 다시 저장하기

5 변수명과 문자

```
print(result)   # 변수 result에 들어 있는 값을 출력한다.
print('result') # 문자 result를 출력한다.
```

6 만들지 않은 변수를 사용하면 `NameError`가 발생한다.

```
naem = '유리'
print(name)
```

```
실행결과
NameError: name 'name' is not defined
```

첫째 줄에서 만든 변수명은 `naem`이므로 `print()`에서 사용한 `name` 변수는 만들어지지 않은 상태다. 이처럼 변수가 만들어지지 않은 상태에서 사용하면 오류가 발생한다.

7 입력받은 값을 변수에 저장하기

```
# 변수명 = input()
r = input('반지름은? ')
```

변수 `r`에 `input()`으로 입력받은 값을 저장한다.

8 변수는 마지막에 저장된 값만 기억하고, 이전에 저장한 값은 기억하지 못한다.

```
abc = 10
abc += 5
abc = 22
print(abc)
```

```
실행결과
22
```

변수 `abc`에 10이 가장 먼저 저장된다. 그다음 줄에서 `abc += 5`를 실행하면 10이 사라지고 대신 15가 저장된다. 마지막으로 `abc = 22`가 실행되면 15는 사라지고 최종적으로 22가 `abc`에 저장된다.

9 데이터 타입 변수가 가진 데이터의 유형이나 형태를 의미한다.

데이터 타입		파이썬 데이터 타입 표기
숫자형	정수형	`int`
	실수형	`float`
문자열		`str`

10 데이터 타입을 잘못 사용해 연산하면 `TypeError`가 발생한다.

11 데이터 타입을 바꾸려면 바꾸려는 데이터 타입으로 데이터를 감싼다.

명령어	의미	예시	설명
`int(a)`	`a`를 정수형으로 바꿔라	`x = 99.9` `x = int(x)`	값이 정수형으로 바뀌어 99를 저장한다.
		`r = int(input())`	입력받은 데이터를 정수형으로 바꿔 `r`에 저장한다. 단, 숫자가 아닌 문자가 입력되면 `ValueError`가 발생한다.
`float(a)`	`a`를 실수형으로 바꿔라	`y = 99` `y = float(y)`	값이 실수형으로 바뀌어 `y`에 99.0을 저장한다.

		`avg = '1.2'`
		`print(avg * 3)`
		`avg = float(avg)`
		`print(avg * 3)`
			`avg` 값은 현재 문자형이므로 처음에 1.21.21.2가 출력된다.
			`avg`를 실수형으로 바꿔 `avg`에 다시 저장한다.
			`avg`가 실수형이므로 3.6이 출력된다.

`str(a)`	`a`를 문자열로 바꿔라	`z = 4.5`
		`z = str(z)`
		`print('학점은 ' + z)`
			`z`를 문자열로 바꾼다.
			문자 + 문자이므로 '학점은 4.5'가 출력된다.

Self Check

1 자신을 소개하는 한 단어를 입력받아 다음처럼 출력되도록 프로그램을 작성해 보세요.

실행결과

```
당신은 누구입니까? 전설
나는 전설이다.
```

입력받는 부분

2 회원가입 프로그램을 만들려고 합니다. 아이디와 이름을 입력받아 다음처럼 출력되도록 프로그램을 작성하세요.

실행결과

입력받는 부분

```
아이디 입력: yoori
이름 입력: 유리
```

```
유리 님! 회원가입을 환영합니다!
yoori 님에게 지금 즉시 사용 가능한 쿠폰 5개 발급!
yoori 님에게만 적립금 2000원 추가 지급!
```

3 다음 프로그램의 입력칸에 '파이썬'을 입력했을 때 실행결과를 예상해 보세요. 정상적으로 실행되면 결괏값을, 오류가 발생한다면 오류가 발생하는 부분과 그 이유를 설명하세요.

```
x = input('단어 입력: ')   # 파이썬 입력
x = int(x)
print(x)
```

힌트 입력칸에 '파이썬'을 입력한다는 것을 염두에 두고 생각해 보세요!

4 국어 16%, 수학 34%, 영어 23%로 반영하는 대학에 입학하고 싶습니다. 국어 점수, 수학 점수, 영어 점수를 각각 입력받아 최종 점수를 다음 예시처럼 출력하는 프로그램을 작성하세요.

실행결과

입력받는 부분

```
국어 점수는? 80
수학 점수는? 60
영어 점수는? 93
```

```
최종 점수는 54.59 입니다!
```

힌트 입력을 몇 개 받아야 하는지, 입력받은 값에 어떤 연산이 필요한지 생각해 보세요!

정답 및 해설